

INT202W11_数论-密码学

Encryption schemes 加密方案

一般思想是，要发送的消息 M （通常称为明文 **plaintext**）被加密 **encrypt** 为无法识别的字符 C 字符串，即密文 **ciphertext**。

然后，密文 C 传输给接收方，接收方解密 **decrypt** C 以恢复原始消息 M 。

Symmetric encryption schemes and secret keys 对称加密方案和密钥

Alice 和 Bob 使用相同的密钥 (K) 来加密和解密消息。

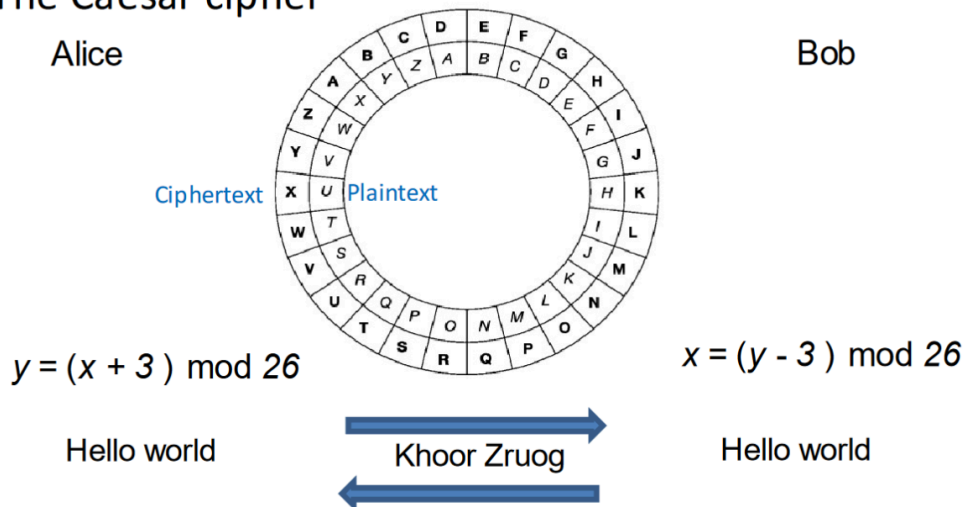
Caesar cipher 凯撒密码

将字母移位为秘密数字（键 K ）：消息中的每个字母都替换为字母表中的另一个字母 k 位置。

$y = (x + k) \bmod n$ ，其中 n 是字母的大小和整数 k ， $1 \leq k < n$ 。

- x = Original letter (A=0, B=1, ..., Z=25).
- k = Secret key (e.g., $k=3$).
- $\bmod 26$ = Wraps around after Z (e.g., $Z + 3 \rightarrow C$).

The Caesar cipher



虽然非常易于使用，但替换密码 **substitution ciphers** 并不安全（Caesar 密码是替换密码之一）

使用频率分析可以很容易地破解替换密码的密钥，该分析基于对所用字母表中各种字母或字母组的频率的了解。

One-time Pad(OTP) 一次一密

对于此加密方案，Alice 和 Bob 共享一个随机位串 K，该字符串的长度与他们将要发送的任何消息一样长。此密钥 K 是用于加密和解密过程的对称密钥。

key 的长度应大于或等于 message key。密钥必须是一次性的，并且每次都会随机生成。

加密消息 M，Alice 计算 $C = M \oplus K$ ，其中 $\oplus$ 符号表示按位“异或 exclusive-or”运算。

$$0 \oplus 0 = 1 \oplus 1 = 0, \quad 0 \oplus 1 = 1 \oplus 0 = 1$$

Key (K): Random bit string, length $\geq$ plaintext

Encrypt: $C = M \oplus K$ (XOR)

Decrypt: $M = C \oplus K$

Public-key cryptography 公钥加密

对称加密方案的一个主要问题是密钥分发，即如何安全地分发密钥。

1976 年，Diffie 和 Hellman 描述了一个克服密钥分发问题的抽象系统——公钥密码系统 Public-key cryptosystems。

由加密函数 E 和解密函数 D 组成。对于任何消息 M，必须满足以下属性：

- $D(E(M)) = M$.
- E 和 D 都很容易计算。
- 从 E 推导出 D 在计算上是不可行的。
- $E(D(M)) = M$.

任何人都可以向函数 D 的持有者发送私人消息，但只有该人知道如何解密它。

因此，E 被称为单向函数 one-way function。E 是公开的，D 保持私密。



Turing's code

明文 M 使用公有密钥 e 和 n 通过以下作进行加密：

$$C = M \times e \bmod n$$

解密再次由模乘法处理：

$$M = C \times d \bmod n$$

若 $d \equiv e^{-1} \pmod n$, 则方案可以传输消息。

但是, 计算模乘逆很容易, 因此此代码不安全, 并且从未使用过。

Desiderata of RSA

明文 M 使用公有密钥 e 和 n 通过以下作进行加密:

$$C = M^e \pmod n$$

解密再次由模幂处理:

$$M = C^d \pmod n$$

我们可以选择 e 、 n 和 d , 使得对于每个整数 $0 \leq x < n$, 我们有 $x^{ed} \equiv x \pmod n$

从 e 和 n 派生 d 在计算上应该是不可行的。

Modular Exponentiation 模幂运算

费马小定理⁺有两种形式:

①若 $\forall a \in \mathbb{Z}$, p 为素数⁺, 则 $a^p \equiv a \pmod p$;

②若 $\forall a \in \mathbb{Z}$, p 为素数, a 不是 p 的倍数 (即 $\gcd(a, p) = 1$), 则 $a^{p-1} \equiv 1 \pmod p$;

(其中, $\gcd(a, p)$ 代表 a 和 p 的最大公约数⁺)

素数 p , 整数 x 满足 $x \pmod p \neq 0$ 则有 $x^{p-1} \equiv 1$

费马小定理有助于计算整数对素数的幂。

◆ Example ($p = 5$):

$$1^4 \pmod 5 = 1$$

$$3^4 \pmod 5 = 81 \pmod 5 = 1$$

$$2^4 \pmod 5 = 16 \pmod 5 = 1$$

$$4^4 \pmod 5 = 256 \pmod 5 = 1$$

推论: 设 p 为素数。对于 $\mathbb{Z}_p^*$ 的每个非零余数 x , x 的乘法逆元是 $x^{p-2} \pmod p$

Euler's totient function 欧拉函数

乘法群 (multiplicative group) $\mathbb{Z}_n^*$ 由所有 $\mathbb{Z}_n$ ($\{0, 1, \dots, n-1\}$) 中所有与 n 互素 (relatively prime) 的数字组成。

Example: $\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$

Euler's totient function $\phi(n)$: 计算 $\mathbb{Z}_n^*$ 的大小

If p is prime: $\phi p = p-1$, $\phi 10 = 4$

The general formula to compute $\phi(n)$

Let the prime factorization of n is given by

$$n = p_1^{e_1} \times p_2^{e_2} \times \cdots \times p_k^{e_k}$$

The Euler's function:

$$\phi(n) = n \times \left(1 - \frac{1}{p_1}\right) \times \left(1 - \frac{1}{p_2}\right) \times \cdots \times \left(1 - \frac{1}{p_k}\right)$$

For examples:

- $15 = 3 \times 5$, $\phi(15) = 15 \times \left(1 - \frac{1}{3}\right) \times \left(1 - \frac{1}{5}\right) = 15 \times \left(\frac{2}{3}\right) \times \left(\frac{4}{5}\right) = 8$
- $12 = 2^2 \times 3$, $\phi(12) = 12 \times \left(1 - \frac{1}{2}\right) \times \left(1 - \frac{1}{3}\right) = 4$

If $n = p \times q$ (p, q are primes)

$$\phi(n) = n \times \left(1 - \frac{1}{p}\right) \times \left(1 - \frac{1}{q}\right) = pq \times \left(\frac{p-1}{p}\right) \times \left(\frac{q-1}{q}\right) = (p-1)(q-1)$$

Theorem [Euler's Theorem] : Let n be a positive integer, and let x be an integer such that $\gcd(x, n) = 1$. Then

$$x^{\phi(n)} \equiv 1 \pmod{n}$$

◆ Example ($n = 10$)

$$3^{\phi(10)} \pmod{10} = 3^4 \pmod{10} = 81 \pmod{10} = 1$$

$$7^{\phi(10)} \pmod{10} = 7^4 \pmod{10} = 2401 \pmod{10} = 1$$

$$9^{\phi(10)} \pmod{10} = 9^4 \pmod{10} = 6561 \pmod{10} = 1$$

|| 计算RSA

Review

RSA Cryptosystem

Setup:

- $n = p \times q$, with p and q primes.
- $\phi(n) = (p-1)(q-1)$
- $\gcd(e, \phi(n)) = 1$
- d of $e^{-1} \bmod \phi(n)$

Keys:

- Public key $K_E = (n, e)$.
- Private key $K_D = d$.

Encryption:

- Plaintext M in Z_n (i.e., $0 \leq M < n$)
- Ciphertext $C = M^e \bmod n$.

Decryption:

- Recover $M = C^d \bmod n$.

Example

Setup:

- $p=7, q=17$
- $n = 7 \times 17 = 119$
- $\phi(119) = 6 \times 16 = 96$
- $e=5$
- $d=77$ ($77 \times 5 = 96 \times 4 + 1$)

Keys:

- Public key $K_E = (119, 5)$.
- Private key $K_D = 77$.

Encryption:

- $M=19$
- $C = 19^5 \bmod 119 = 66$

Decryption:

- $M = 66^{77} \bmod 119 = 19$. 2

The RSA method

Complete RSA Example

Setup:

- $p=5, q=11$
- $n = 5 \times 11 = 55$
- $\phi(55) = 4 \times 10 = 40$
- $e=3$
- $d=27$ ($27 \times 3 = 40 \times 2 + 1$)

Encryption:

- $C = M^3 \bmod 55$

Decryption:

- $M = C^{27} \bmod 55$

M	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C	1	8	27	9	15	51	13	17	14	10	11	23	52	49	20	26	18	2
M	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
C	39	25	21	33	12	19	5	31	48	7	24	50	36	43	22	34	30	16
M	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
C	53	37	29	35	6	3	32	44	45	41	38	42	4	40	46	28	47	54

22

RSA 算法中的一个可能的瓶颈是计算 $x^k \bmod n$

Fast exponentiation 快速幂计算

任何数字都可以表示为 2 的不同幂之和。

$$3^{94} \pmod{17}$$

Any number can be represented as the sum of distinct powers of two.

Use the smallest numbers whether they are positive or negative

$$94 = 64 + 16 + 8 + 4 + 2$$

$$3^2 \equiv 9 \pmod{17}$$

mod 17

$$3^4 \equiv 81 \equiv 13 \equiv -4 \pmod{17}$$

$$3^8 \equiv (3^4)^2 \equiv 16 \equiv -1 \pmod{17}$$

$$3^{16} \equiv (3^8)^2 \equiv (-1)^2 \equiv 1 \pmod{17}$$

$$3^{64} \equiv (3^{16})^4 \equiv (1)^4 \equiv 1 \pmod{17}$$

$$3^{94} \equiv 3^{64+16+8+4+2} \pmod{17}$$

$$\equiv 3^{64} 3^{16} 3^8 3^4 3^2 \pmod{17}$$

$$\equiv (1)(1)(-1)(-4)(9) \pmod{17}$$

$$\equiv 36 \pmod{17}$$

$$\equiv 2 \pmod{17}$$

使用基于 “repeated squaring” 的算法，我们可以做得更好。

compute x^{16} , we could first find x^2 , then $(x^2)^2 = x^4$, then $(x^4)^2 = x^8$, and finally $(x^8)^2 = x^{16}$.

这只需要四次乘法，而不是 “naïve method” 的 15 次乘法。

复杂度是 $O(\log k)$

|| 手算RSA

加密: $C = M^e \bmod n$.
解密: $M = C^d \bmod n$.
 $x^{ed} \equiv x \pmod{n}$.
 $n = p \times q$ (给选, 2个素数)
 $\phi(n) = (p-1)(q-1)$ (欧拉函数) $\rightarrow n = p \times q = 7 \times 17 = 119$
 $\phi(119) = 96$.
 $\gcd(e, \phi(n)) = 1$. $e = 5$ (随便找一个互素的 e)
 $d = e^{-1} \bmod \phi(n)$ $d = 5^{-1} \bmod 96 = 77$.
 $5d \equiv 1 \bmod 96$.
 $96 \times 4 = 384$.
 $d = 77$
 $\Rightarrow \begin{cases} \text{Pub key } K_E = (119, 5) \\ \text{Pri key } K_D = 77 \end{cases}$
Encryption: $M = 19$

$C = 19^5 \bmod 119 = 66$.
 $M = 66^{77} \bmod 119 = 19$
(计算快速幂)
 $77 = 64 + 8 + 4 + 1$
 $\begin{pmatrix} 2^6 & 2^3 & 2^2 & 2^0 \\ 1 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$
 $66^1 \bmod 119 = 66$.
 $66^4 \equiv (66^2)^2 \equiv (72)^2 \bmod 119 = 67$.
 $66^8 \equiv (66^4)^2 \equiv (72)^4 \bmod 119 = 86$.
 $66^{64} \equiv (66^{32})^2 \equiv (72)^{32} \equiv (72^4)^8 \equiv (86)^8$
 $\equiv (18)^4 = 18$.
 $= (66)(67)(86)(18) \bmod 119$.
 $= 19 = M$

Digital signatures 数字签名

RSA 密码系统支持数字签名。假设 Bob 向 Alice 发送了一条消息 M，并且 Alice 想要验证该消息是 Bob 发送的。Bob 可以使用应用于 M 的解密函数创建签名：

$$S = M^d \bmod n$$

Alice 使用加密功能验证数字签名，即检查

$$M \equiv S^e \bmod n$$

随着因式分解较大数的能力增加，我们只需要选择更大的素数 p 和 q，这样 n =pq 就超出了当前的因式分解能力。

RSA 依赖于这样一个事实，即因式分解大数字非常困难。

Estimated resources needed to factor a number within one year

Bits	PCs	Memory
430	1	128MB
760	215,000	4GB
1,020	342×10 ⁶	170GB
1,620	1.6×10 ¹⁵	120TB